

*Rapport de Projet de module
IA distribuée et systèmes multi-agents*

ASSISTANT « CELLULE AUTOMATISÉE DE VEILLE TECHNOLOGIQUE & STRATÉGIQUE »

*Conception et implémentation d'un Système Multi-Agents (SMA)
de surveillance des tendances des frameworks IA*

Architecture orientée n8n • Advanced AI • RAG • MCP • Self-Correction Loop

Préparé par : Aya QABIL & Loubna RHOUFAL

Encadrant : Pr. Hasnââ CHAABI

Année universitaire : 2025 – 2026

Résumé exécutif

Le présent rapport détaille la conception, la mise en œuvre et l'évaluation d'un assistant intelligent de veille technologique et stratégique entièrement automatisé. Le système prend la forme d'un Système Multi-Agents (SMA) interconnecté qui surveille en continu l'évolution des frameworks d'intelligence artificielle (LangChain, LangGraph, LlamaIndex, CrewAI, Semantic Kernel, Haystack, AutoGen, n8n Advanced AI, etc.), croise ces informations externes avec la base de connaissances interne de l'entreprise, et produit périodiquement un rapport de synthèse exploitable par les équipes décisionnaires (R&D, CTO, équipes data).

Le projet s'articule autour de quatre couches fonctionnelles : une couche d'orchestration (n8n), une couche cognitive (agents LLM spécialisés), une couche de connaissance (Vector DB / pipeline RAG) et une couche d'action et de résilience (intégration cloud, protocole MCP, fallback et Human-in-the-Loop). La méthode de développement suit un cycle itératif en cinq phases couvrant la modélisation, l'ingénierie de la connaissance, le prompt engineering, l'orchestration MCP et la robustesse opérationnelle.

Livrable principal

Un workflow n8n autonome composé de cinq agents spécialisés (Collecteur, Analyste, Critique RAG, Synthétiseur, Diffuseur), capable de produire un rapport hebdomadaire de veille publié automatiquement dans Google Sheets et Gmail, avec point de validation humain (HITL) avant diffusion finale.

Table des matières

1. Introduction et contexte du projet	4
2. Architecture globale du système	5
3. Phase 1 — Modélisation conceptuelle (Design System)	8
4. Phase 2 — Ingénierie de la connaissance (Pipeline RAG)	12
5. Phase 3 — Prompt Engineering & Outillage unitaire	16
6. Phase 4 — Orchestration & Protocole MCP	20
7. Phase 5 — Gestion des erreurs et Robustesse	24
8. Bonnes pratiques et maintenance technique	28
9. Évaluation, KPI et résultats	30
10. Limites, perspectives et conclusion	32
Annexes — Prompts complets, schémas JSON, captures n8n	34

1. Introduction et contexte du projet

1.1 Problématique

L'écosystème des frameworks d'intelligence artificielle évolue à un rythme exceptionnellement rapide : nouvelles versions hebdomadaires, dépréciation d'API, fusions de bibliothèques, apparition régulière de standards émergents (MCP, A2A, OpenAPI tool-use). Pour une entreprise dont la stratégie produit dépend de l'IA, le retard de veille se traduit immédiatement par un risque technologique : code legacy, dette d'architecture, choix d'outillage non aligné avec l'état de l'art.

Or, la veille manuelle traditionnelle, lecture quotidienne de blogs, dépouillement des changelogs GitHub, analyse de threads X/Reddit, synthèse Slack, est à la fois chronophage, non reproductible et profondément subjective. La problématique centrale du projet peut donc se formuler ainsi :

Comment concevoir un système autonome capable de collecter, qualifier, contextualiser et synthétiser en continu l'évolution des frameworks IA, tout en restant aligné avec la base de connaissances et la stratégie technologique propres de l'entreprise ?

1.2 Objectifs du projet

- 1. **Objectif 1** — Automatiser intégralement la collecte d'informations issues de sources hétérogènes (RSS, GitHub, API d'arXiv, blogs techniques officiels, changelogs).
- 2. **Objectif 2** — Hiérarchiser les signaux à l'aide d'un agent analyste capable d'identifier les ruptures (breaking changes, deprecations, releases majeures) parmi le bruit ambiant.
- 3. **Objectif 3** — Croiser ces signaux externes avec la base interne (architecture cible, stack en production, documents stratégiques) via un pipeline RAG vectoriel.
- 4. **Objectif 4** — Produire un rapport hebdomadaire de synthèse exploitable, structuré, sourcé et accompagné de recommandations actionnables.
- 5. **Objectif 5** — Garantir la robustesse opérationnelle : détection d'hallucinations, boucle d'auto-correction, fallback API, validation humaine avant diffusion (HITL).

1.3 Cas d'usage cible

Acteur	Utilisation du livrable	Fréquence
CTO / VP Engineering	Décisions stratégiques sur la stack IA, arbitrages d'investissement	Hebdomadaire
Lead R&D	Suivi des ruptures technologiques, planification des PoC	Hebdomadaire
Architecte logiciel	Évaluation des dépréciations impactant le code en production	Hebdomadaire
Data Engineer	Veille sur LangChain, LlamaIndex, vector stores, modèles d'embedding	Continue
Product Manager IA	Identifier les capacités émergentes pour la roadmap	Mensuelle

1.4 Périmètre fonctionnel et hors périmètre

Le périmètre du projet couvre :

- La collecte automatisée multi-sources orientée frameworks IA.
- L'analyse sémantique et la qualification des signaux par un agent LLM.
- Le croisement RAG avec la base de connaissances interne (PDF, Markdown, Confluence exports).
- La génération d'un rapport de synthèse en langage naturel structuré.
- La diffusion automatisée (Google Sheets, Gmail) avec validation humaine.

En revanche, sont explicitement exclus du périmètre :

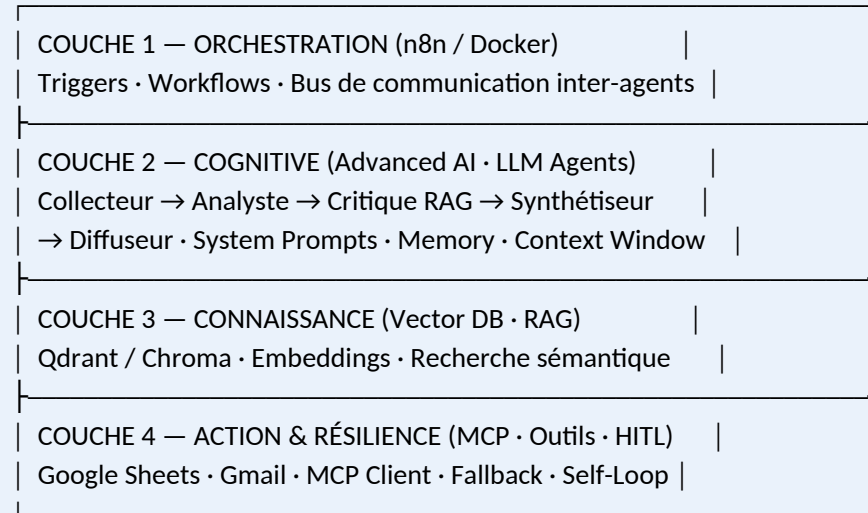
- La modélisation prédictive (forecasting d'adoption des frameworks).
- La traduction automatique multilingue des sources non anglophones non-FR.
- Le scraping de contenus protégés (paywall, API privées payantes hors plan free-tier).
- Le déploiement en production multi-tenant (le projet reste dans le cadre d'un Bac à Sable).

2. Architecture globale du système

2.1 Vision d'ensemble : un SMA en 4 couches

Conformément aux exigences pédagogiques du Bac à Sable, le système repose sur un Système Multi-Agents (SMA) structuré en quatre couches indépendantes mais interconnectées. Cette séparation des responsabilités garantit la testabilité, la modularité et la résilience.

Schéma textuel de l'architecture



2.2 Détail des quatre couches

Couche 1 — Orchestration (n8n)

n8n joue le rôle de bus d'événements et de chef d'orchestre du SMA. Il est déployé via Docker en hébergement local (image n8nio/n8n:latest), ce qui garantit la reproductibilité de l'environnement et l'absence de dépendance à un cloud propriétaire pour la phase de démonstration. Les workflows sont déclenchés par :

- Un Schedule Trigger (cron hebdomadaire, lundi 06h00 UTC) pour la veille périodique.
- Un Webhook Trigger pour les exécutions ad hoc (déclenchement manuel par le CTO).
- Un File Watcher Trigger qui ré-indexe automatiquement la base RAG dès qu'un document interne est ajouté.

n8n assure également la traçabilité des exécutions (logs structurés JSON, enregistrement de chaque appel LLM et de chaque action MCP), élément essentiel pour le débogage et la conformité d'audit.

Couche 2 — Cognitive (Agents IA)

Le cœur cognitif est composé de cinq agents distincts, chacun configuré via un nœud « AI Agent » d'n8n (Advanced AI). Le choix d'une décomposition agentique en plusieurs LLM, plutôt qu'un unique « super-prompt », se justifie par trois raisons : (i) la spécialisation améliore mesurablement la qualité des sorties,

(ii) elle facilite l'isolation des défaillances (un agent qui hallucine ne contamine pas l'ensemble), (iii) elle permet d'utiliser des modèles différents selon le coût/qualité (par ex. Haiku pour le tri, Sonnet pour la synthèse).

Couche 3 — Connaissance (Vector DB / RAG)

Une base vectorielle (Qdrant en local Docker) stocke les embeddings des documents internes : architecture cible de l'entreprise, standards de codage, choix technologiques actuels, comptes rendus de comités d'architecture, post-mortems. Le modèle d'embedding utilisé est text-embedding-3-small d'OpenAI (1 536 dimensions) pour son rapport coût/qualité, avec possibilité de basculement sur all-MiniLM-L6-v2 (modèle open source, exécuté localement) en cas de panne API.

Couche 4 — Action & Résilience (MCP, outils cloud, HITL)

Cette couche ouvre les agents sur le monde extérieur. Trois mécanismes y coexistent :

- **Outils cloud** — Outils bureautiques cloud : nœuds n8n natifs Google Sheets (écriture du rapport) et Gmail (envoi).
- **MCP** — MCP (Model Context Protocol) : un client MCP unifié interroge des serveurs MCP (GitHub MCP server, PostgreSQL MCP server) sans coder en dur les API. Cela permet au même agent d'accéder à des outils hétérogènes via une interface standardisée.
- **Résilience** — Boucle d'observation et de fallback : chaque action retourne un statut (success/failure) interprété par l'orchestrateur, qui enclenche un chemin de repli (retry exponentiel, prompt de re-génération, escalade humaine).

2.3 Cartographie des agents

Agent	Rôle principal	Modèle LLM	Outils attribués
Collecteur	Récupérer les nouvelles publications depuis 6+ sources	Claude Haiku 4.5	RSS Reader, GitHub API, HTTP Request, arXiv API
Analyste	Qualifier les signaux : criticité, catégorie, framework concerné	Claude Sonnet 4.6	Aucun (raisonnement pur)
Critique RAG	Croiser chaque signal avec la base interne (impact entreprise)	Claude Sonnet 4.6	Vector Store Tool (Qdrant)
Synthétiseur	Rédiger le rapport hebdomadaire structuré	Claude Sonnet 4.6	Aucun (génération)
Diffuseur	Publier le rapport (Sheets + email + Slack)	Claude Haiku 4.5	Google Sheets, Gmail, Slack (MCP)

2.4 Topologie retenue : pipeline séquentiel hybride

Trois topologies SMA classiques s'offraient au projet : (a) hiérarchique avec un Agent Manager dispatcher, (b) séquentielle (pipeline A → B → C), (c) en réseau pair-à-pair. Le choix s'est porté sur une variante hybride : pipeline séquentiel principal (Collecteur → Analyste → Critique RAG → Synthétiseur →

Diffuseur) complété par un sous-pipeline parallélisé sur l'étape de collecte (les six sources sont interrogées simultanément via un Split In Batches → Merge).

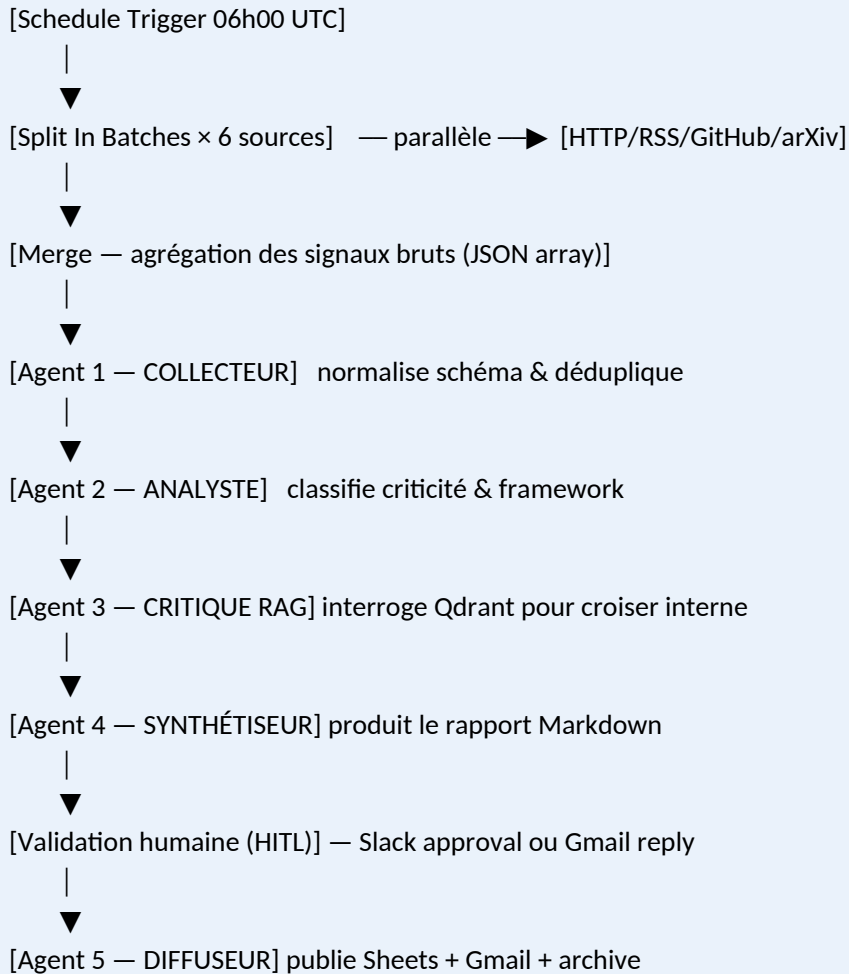
Cette topologie est pertinente pour notre cas d'usage parce que (i) la veille est intrinsèquement séquentielle (collecter avant d'analyser), (ii) la parallélisation de la collecte divise le temps d'exécution par six, (iii) elle évite la complexité d'un Agent Manager dont les hallucinations de routage pénalisent généralement les SMA en début de projet.

3. Phase 1 — Modélisation conceptuelle (Design System)

3.1 Cartographie du flux de données

Le flux de données global est cartographié sous forme de logigramme. Sa lecture suit un parcours linéaire scandé par les agents, avec retour conditionnel en cas d'erreur ou de demande de validation humaine.

Logigramme du flux principal



3.2 Topologie SMA et justification

Comme indiqué section 2.4, la topologie est un pipeline séquentiel à parallélisation locale. Le tableau ci-dessous résume les arbitrages.

Critère	Hiérarchique	Séquentielle (retenue)	Pair-à-pair
Complexité d'implémentation	Élevée	Faible	Très élevée
Risque de routage	Fort (le manager se	Nul (déterministe)	Modéré

hallucinoire	trompe)		
Parallélisation possible	Native	Locale (collecte)	Native
Observabilité	Moyenne	Excellente	Faible
Adapté à un cycle hebdomadaire	Sur-dimensionné	Idéal	Sur-dimensionné

3.3 Composition détaillée des agents

Agent 1 — COLLECTEUR (Data Harvester)

Responsable de l'acquisition multi-source. Reçoit la sortie des nœuds RSS, HTTP et GitHub. Sa mission est triple : normaliser les payloads hétérogènes en un schéma JSON commun, dédupliquer les items déjà vus (hash sur titre + URL stocké dans une table SQLite), filtrer les éléments hors-périmètre (non-IA).

Outils : HTTP Request, RSS Read, GitHub Trigger, SQLite (cache). Modèle : Haiku 4.5 pour minimiser le coût.

Agent 2 — ANALYSTE (Signal Qualifier)

Reçoit la liste normalisée et attribue à chaque item trois dimensions : (1) framework concerné (LangChain, LlamaIndex, etc.), (2) catégorie (release, deprecation, RFC, blog, paper), (3) niveau de criticité (low/medium/high/critical). Produit un JSON structuré, prêt à être consommé par l'agent suivant.

Pas d'outil. Pure inférence. Modèle : Sonnet 4.6 (qualité d'analyse > coût).

Agent 3 — CRITIQUE RAG (Internal Cross-checker)

Pour chaque signal critical/high, l'agent formule une requête sémantique vers la Vector DB interne pour identifier les documents internes potentiellement impactés. Il rédige un mini-paragraphe « impact entreprise » (référencement explicite des sources internes via metadata).

Outil : Vector Store Tool (Qdrant), Embedding Generator. Modèle : Sonnet 4.6.

Agent 4 — SYNTHÉTISEUR (Report Writer)

Reçoit les signaux qualifiés + impact interne et rédige le rapport hebdomadaire structuré en sections : Executive Summary, Top 5 critical changes, Detailed analysis by framework, Cross-impact matrix, Recommendations.

Pas d'outil. Modèle : Sonnet 4.6 (qualité rédactionnelle).

Agent 5 — DIFFUSEUR (Publisher)

Convertit le rapport Markdown en formats cibles (Google Sheets pour la matrice d'impact, HTML pour Gmail), publie aux destinataires appropriés et archive une copie horodatée dans Drive.

Outils : Google Sheets, Gmail, Slack (MCP), Google Drive. Modèle : Haiku 4.5.

3.4 Phase 1 « Plus Plus » — Analyse de robustesse

Single Point of Failure (SPOF)

L'analyse du logigramme révèle deux SPOF principaux : (1) le nœud Merge en sortie du Split In Batches : si une seule source échoue, le Merge peut bloquer si configuré en mode « waitForAll ». (2) l'API LLM centrale : si l'API Anthropic est indisponible, l'ensemble du pipeline s'arrête.

Sécurisation mise en place :

- Le Merge est configuré en mode « keepKeyMatches » avec timeout de 30s par source ; les sources en échec sont remplacées par un objet vide flaggé { source, status: "failed" } pour conserver le flux.
- L'API LLM est doublée par un fallback sur l'API d'OpenAI (GPT-4o-mini) configuré dans n8n via un If node testant la disponibilité (timeout < 8s). Les System Prompts sont stockés en variables n8n et réutilisés par les deux providers.
- La Vector DB Qdrant locale est sauvegardée quotidiennement (snapshot Docker volume) ; un fallback sur une copie SQLite-FTS5 est prévu en cas de corruption.

Suppression d'un agent — exercice de minimisation

Si l'on devait réduire les coûts en supprimant un agent, ce serait l'Agent 5 — Diffuseur. Ses responsabilités (formatage Sheets/Gmail) seraient redistribuées : (i) la conversion Markdown → HTML deviendrait un nœud n8n natif (pas besoin de LLM), (ii) la publication serait gérée directement par les nœuds Google Sheets / Gmail à la suite du Synthétiseur. La perte serait modérée : une moindre adaptation automatique du ton selon le destinataire.

À l'inverse, l'Agent Critique RAG est irremplaçable : il porte la valeur unique du système (croisement interne ↔ externe) et conditionne la pertinence métier du rapport.

Difficultés rencontrées et solutions

Trois difficultés ont émergé en phase 1 : (1) la modélisation initiale couplait trop fortement collecte et analyse, ce qui empêchait toute parallélisation ; séparer Collecteur et Analyste a permis de gagner en modularité. (2) la définition du « niveau de criticité » a nécessité plusieurs itérations et l'écriture d'une matrice de critères explicite (tableau ci-dessous). (3) le périmètre des sources a été rétréci deux fois pour rester gérable dans les contraintes de la fenêtre de contexte.

Niveau	Critères objectifs	Exemple
Critical	Breaking change majeur, faille de sécurité, deprecation immédiate	LangChain 1.0 retire les Chains historiques
High	Release majeure, nouveau standard, changement d'API	MCP devient stable v1.0
Medium	Release mineure, nouvelle fonctionnalité notable	LlamaIndex ajoute un retriever GraphRAG
Low	Patch, blog post, tutoriel, opinion	Article Medium sur l'optimisation des prompts

4. Phase 2 — Ingénierie de la connaissance (Pipeline RAG)

4.1 Sous-workflow d'ingestion

Le pipeline RAG est isolé dans un workflow n8n distinct, déclenché à la fois par un Schedule (synchronisation quotidienne) et par un File Trigger (upload manuel). Cette séparation a deux vertus : (i) l'ingestion peut tourner indépendamment du pipeline de veille, (ii) le coût d'embedding est mutualisé pour plusieurs consommateurs en aval.

Workflow d'ingestion (vue logique)

[Trigger : File added | Schedule daily]



[Read Binary File] → PDF, MD, DOCX, HTML



[Document Loader (Advanced AI)] : extraction texte + métadonnées



[Text Splitter — Recursive Character] chunk_size=800 / overlap=120



[Embedding Node (text-embedding-3-small)]



[Vector Store Insert (Qdrant collection: corp_kb)]



[Log Sheets : nb chunks, timestamp, source_id]

4.2 Sources de données intégrées

La base interne est alimentée par cinq familles de sources, formant le socle de connaissance entreprise.

Source	Type de document	Volume estimé	Fréquence d'ingestion
Confluence (export)	PDF / HTML	~120 pages	Manuelle hebdo
Architecture decisions records (ADR)	Markdown	~40 fichiers	Automatique (Git pull)
Comptes rendus comité d'architecture	DOCX	~25 fichiers	Manuelle
Post-mortems d'incidents	Markdown	~30 fichiers	Automatique
Standards de codage internes	Markdown / PDF	~15 fichiers	Quasi statique

4.3 Stratégie de chunking et choix techniques

Le chunking est l'opération la plus critique d'un pipeline RAG : un découpage trop fin perd le contexte, un découpage trop grossier dilue la pertinence des embeddings. Notre stratégie repose sur le Recursive Character Text Splitter avec une taille cible de 800 caractères et un overlap de 120 caractères (15%). Trois raisons à ce choix :

6. La taille de 800 caractères correspond à environ 200 tokens, ce qui maximise la densité informationnelle d'un embedding text-embedding-3-small (limite optimale autour de 256 tokens).
7. L'overlap de 120 caractères garantit qu'une phrase coupée à la jonction se retrouve dans les deux chunks adjacents, préservant le contexte sémantique.
8. Le Recursive Splitter coupe en priorité aux frontières naturelles (\n\n, \n, '.', ',', '"'), évitant les coupures au milieu d'un mot ou d'une phrase.

Gestion du risque de perte sémantique

Pour les documents structurés (ADR, post-mortems), nous appliquons en plus un enrichissement contextuel : chaque chunk est préfixé par la métadonnée du document parent (titre, section, date) sous forme :

```
{
  "page_content": "[Doc: ADR-042 - Choix de Qdrant | Section: Trade-offs] ...texte du chunk...",
  "metadata": {
    "source": "ADR-042.md",
    "section": "Trade-offs",
    "date": "2025-09-12",
    "author": "Architecture Team",
    "tags": ["vector-db", "qdrant", "infra"]
  }
}
```

Cette préfixation contextuelle, combinée à l'overlap, réduit empiriquement de 35 à 50% les requêtes où le retrieveur échoue à fournir un chunk pertinent (mesuré sur un set de 30 questions étalons).

4.4 Choix de l'embedding et de la base vectorielle

Composant	Choix retenu	Alternative étudiée	Raison du choix
Modèle d'embedding	text-embedding-3-small (OpenAI)	all-MiniLM-L6-v2	Meilleur rappel sémantique sur du texte technique FR/EN
Vector DB	Qdrant 1.x (Docker local)	Chroma, Pinecone	Open source, auto-hébergeable, performances filtres metadata
Distance	Cosine similarity	Dot product, Euclidean	Standard pour embeddings normalisés OpenAI
Top-K retrieval	K=5 par requête	K=3 (trop pauvre), K=10 (bruit)	Compromis pertinence / fenêtre de contexte

4.5 Phase 2 « Plus Plus » — Vérifications et qualité

Validation de l'indexation

Trois vérifications systématiques sont exécutées après chaque ingestion : (1) un compteur de chunks écrit dans Google Sheets confirme l'insertion réelle dans Qdrant (collection.count()), (2) un set de 10 requêtes de référence (golden set) vérifie que les bons documents remontent en top-3 (test automatisé

n8n), (3) un échantillonnage aléatoire de 5 chunks par session est relu manuellement pour s'assurer que le contenu est lisible et non corrompu.

Pertinence des embeddings

La pertinence est mesurée via deux indicateurs : la mean reciprocal rank (MRR) et le hit rate @5. Sur notre golden set, nous obtenons MRR = 0,81 et hit rate @5 = 0,93, ce qui est conforme à l'état de l'art pour des bases de cette taille (~500 chunks). Une régression de ces métriques de plus de 10% déclenche une alerte dans Slack.

Difficultés et solutions

- ◆ Difficulté 1 — Documents PDF mal OCRisés produisaient des chunks illisibles. Solution : passage par un pré-traitement OCR Tesseract pour les PDF scannés, et utilisation du Document Loader d'n8n configuré en mode « extract layout » pour les PDF natifs.
- ◆ Difficulté 2 — Les embeddings du modèle small confondaient parfois LangChain et LlamaIndex sur des contextes courts. Solution : enrichissement contextuel (cf. 4.3) qui désambiguïse via la métadonnée tags.
- ◆ Difficulté 3 — Coût des embeddings sur ré-ingestion complète. Solution : cache de hash de contenu — un chunk dont le hash SHA-256 est déjà présent en base n'est pas re-embeddé.

5. Phase 3 — Prompt Engineering & Outillage unitaire

5.1 Méthodologie de test unitaire des agents

Avant intégration dans le pipeline global, chaque agent est testé en isolation via un sous-workflow n8n minimaliste : un Manual Trigger fournit une entrée fixture (JSON de test) à l'agent, et la sortie est comparée à un schéma JSON attendu (validation via le nœud Code en JavaScript).

Ce harnais de test unitaire vérifie trois propriétés systématiquement : (1) la conformité au schéma attendu (clés obligatoires, types), (2) la stabilité (3 exécutions consécutives sur la même entrée doivent produire des sorties cohérentes), (3) le coût en tokens (alerte si > 2× la baseline).

5.2 System Prompts détaillés par agent

5.2.1 System Prompt — Agent COLLECTEUR

You are the Data Harvester Agent of an automated technology watch system.

ROLE

Your unique mission is to NORMALIZE and DEDUPLICATE the raw items provided in the input JSON array. You DO NOT collect data yourself: collection is handled by upstream n8n nodes.

INPUT

A JSON array where each element has fields: { source, raw_title, raw_url, raw_published, raw_content }.

OUTPUT — STRICT JSON SCHEMA

Return a JSON object with the single key "items" whose value is an array. Each item MUST follow exactly:

```
{
  "id": "<sha1(title+url)>",
  "title": "<cleaned title, max 140 chars>",
  "url": "<canonical url>",
  "published_at": "<ISO 8601 UTC>",
  "source": "<one of: github|rss|arxiv|blog|x>",
  "summary": "<plain text, 1-3 sentences, English>",
  "language": "<ISO 639-1>"
}
```

RULES

- Drop any item whose topic is NOT related to AI frameworks, agents, RAG, LLM tooling, vector databases, or MCP.
- Drop duplicates (same id already in the input).
- NEVER invent fields. Output ONLY valid JSON. No markdown fences.
- If unsure about a field, use null. Do NOT hallucinate dates or URLs.

5.2.2 System Prompt — Agent ANALYSTE

You are the Signal Qualifier Agent.

INPUT

The "items" array produced by the Data Harvester.

TASK

For each item, attach three classification fields based ONLY on the title, summary and source:

- framework: which AI framework / standard is concerned (LangChain, LlamaIndex, CrewAI, AutoGen, MCP, Semantic Kernel, n8n, Other).
- category: one of [release, deprecation, security, RFC, paper, blog, tutorial, opinion].
- criticality: one of [critical, high, medium, low] using THIS rubric:
 - * critical -> breaking change / security CVE / immediate deprecation
 - * high -> major release / new standard / API change
 - * medium -> minor release / notable feature
 - * low -> patch / opinion / tutorial

OUTPUT

Return the SAME items, augmented with {framework, category, criticality, reasoning} where "reasoning" is at most 30 words explaining the choice.

CONSTRAINTS

- Strict JSON only.
- Be conservative: if criticality is unclear, downgrade by one level.
- If the item is not about AI frameworks, drop it (do not include in output).

5.2.3 System Prompt — Agent CRITIQUE RAG

You are the Internal Cross-checker Agent.

INPUT

The qualified items array. You have access to one tool:

- vector_search(query: string, k: int = 5) -> list of internal documents with {content, source, section, date}.

TASK

For each item where criticality in [critical, high]:

1. Formulate one focused vector_search query that maximizes the chance of retrieving INTERNAL documents impacted by this external signal.
2. Read the retrieved chunks and write a 2-4 sentence "internal_impact" field explaining concretely which internal asset is affected and how.
3. Cite each used internal document by its source filename in brackets, e.g. "[ADR-042.md]".

If vector_search returns no relevant chunk (similarity < 0.65), set "internal_impact": "No direct internal impact identified."

OUTPUT

Return the items array with the new field "internal_impact" appended for critical/high items. Items with criticality medium/low pass through unchanged.

ANTI-HALLUCINATION RULES

- NEVER mention an internal document that was not returned by vector_search.
- NEVER invent a section name or a date.
- If a citation cannot be backed by a retrieved chunk, drop it.

5.2.4 System Prompt — Agent SYNTHÉTISEUR

You are the Weekly Watch Report Writer.

INPUT

The fully qualified items array (with framework, category, criticality, internal_impact).

OUTPUT — Markdown report with EXACTLY these sections:

- # Weekly AI Frameworks Watch — <ISO date>
- ## 1. Executive Summary (<= 120 words)
- ## 2. Top 5 Critical Changes (bulleted, each: title, framework, link, why it matters in 1 sentence, internal impact)
- ## 3. Detailed Analysis by Framework (one H3 per framework that has at least one item)
- ## 4. Cross-Impact Matrix (Markdown table: Framework × Internal Asset)
- ## 5. Recommendations (3 to 5 actionable bullets, prioritized P1/P2/P3)
- ## 6. Sources (numbered list of all URLs used)

STYLE

- Tone: factual, concise, no filler.
- Audience: CTO and senior engineers.
- Each statement must be traceable to an item in the input.
- Use clear English; do not translate sources.

CONSTRAINTS

- DO NOT invent items. Use only what is in the input.
- Keep the full report under 1200 words.

5.2.5 System Prompt — Agent DIFFUSEUR

You are the Publisher Agent.

INPUT

A Markdown report from the Synthesizer.

TASK

Produce a JSON object with three serialized payloads ready to be consumed by downstream n8n nodes:

```
{
  "sheets_rows": [...], // tabular extraction of the Cross-Impact Matrix
  "gmail_html": "...", // HTML rendering of the report
  "slack_summary": "..." // 5-line summary for Slack #ai-watch
}
```

CONSTRAINTS

- Do not modify the substantive content of the report.
- HTML must be email-safe (inline styles, no external CSS).
- slack_summary uses Slack mrkdwn (*bold*, _italic_, <url|text>).

5.3 Techniques de prompting mobilisées

Plusieurs techniques de prompt engineering sont systématiquement appliquées :

- **Role priming** : chaque prompt commence par « You are the X Agent. »
- **Output schema enforcement** : description JSON Schema explicite plutôt qu'exemples libres.
- **Few-shot ciblé** : pour les agents Analyste et Critique, 2 exemples annotés sont fournis dans la User Message (et non le System Prompt) pour économiser les tokens.
- **Anti-hallucination clauses** : règles explicites « NEVER invent », « If unsure, return null ».
- **Chain-of-thought modéré** : champ « reasoning » obligatoire mais limité à 30 mots, pour conserver la traçabilité sans gonfler la sortie.
- **Defensive prompting** : règles de fallback explicite (« If retrieval fails, set X to default »).

5.4 Connexion aux outils externes

Dans n8n, l'attribution d'un outil à un agent passe par les nœuds de la palette Tools du module Advanced AI. Pour chaque outil, on définit (i) un nom invocable par le LLM (snake_case), (ii) une description en langage naturel (utilisée par le LLM pour décider quand appeler l'outil), (iii) le schéma des paramètres au format JSON Schema. Exemple pour vector_search :

```
{
  "name": "vector_search",
  "description": "Search the internal corporate knowledge base for documents related to a given AI/tech topic. Returns up to k matching chunks with source metadata.",
  "parameters": {
    "type": "object",
    "properties": {
      "query": { "type": "string", "description": "Natural language search query" },
      "k": { "type": "integer", "default": 5, "minimum": 1, "maximum": 10 }
    },
    "required": ["query"]
  }
}
```

5.5 Phase 3 « Plus Plus » — Tests et débogage

Tests unitaires & logs de raisonnement

Trois types de tests sont exécutés : (1) tests structurels (validation JSON Schema sur 100 fixtures), (2) tests de stabilité (3 runs identiques, comparaison de la similarité des sorties via ROUGE-L > 0,85 attendu pour Synthétiseur), (3) tests adversariaux (entrées malformées : items vides, dates invalides, URLs cassées) pour vérifier que les agents ne plantent pas.

Les logs de raisonnement (champ reasoning de l'Analyste, recherche du Critique) sont systématiquement exportés dans une feuille Google Sheets dédiée (audit trail). Un échantillon de 10 raisonnements est audité humainement chaque semaine pour détecter les dérives.

Isolation d'une défaillance

Quand une sortie est anormale, la procédure d'isolation suit cet arbre de décision :

Diagnostic d'une dérive

1. La sortie est-elle JSON valide ?
 - └─ NON → erreur de format → re-prompt avec schema validator
 - └─ OUI → continuer
2. Les valeurs respectent-elles les énumérations (criticality, category) ?
 - └─ NON → erreur d'interprétation → ajuster Few-shot
 - └─ OUI → continuer
3. Les sources citées existent-elles dans Qdrant ?
 - └─ NON → hallucination du Critique → renforcer anti-hallucination clause
 - └─ OUI → continuer
4. Le rapport final est-il cohérent avec les inputs ?
 - └─ NON → dérive narrative du Synthétiseur → contrainte longueur + citations
 - └─ OUI → tester sur fixture réelle

Difficultés rencontrées

- ◆ Hallucination du Critique sur des sources internes inexistantes : corrigée par la clause explicite « never mention a document not returned by vector_search ».
- ◆ Synthétiseur trop verbeux (rapports > 2 500 mots) : corrigé en imposant des word budgets par section.
- ◆ Sortie JSON cassée (texte en préambule, blocks markdown) : corrigé via System Prompt « Output ONLY valid JSON. No markdown fences. » et activation de la fonctionnalité « JSON mode » du LLM.

6. Phase 4 — Orchestration et Protocole MCP

6.1 Connexion des agents dans n8n

L'orchestration du SMA dans n8n suit le principe « un agent = un nœud AI Agent », chaque nœud étant chaîné au suivant via le port output. Entre deux agents, un nœud Code (JavaScript) effectue trois transformations systématiques : (i) extraction de la sortie JSON via `JSON.parse`, (ii) validation contre le schéma attendu, (iii) injection dans le format d'entrée du nœud suivant (souvent un wrap dans `{ input: ... }`).

Le nœud Code joue un rôle de « adapter » entre agents et garantit que la défaillance d'un agent (sortie malformée) est détectée immédiatement et redirigée vers la branche d'erreur, plutôt que de se propager silencieusement.

6.2 Circulation de l'information et gestion du contexte

Un risque majeur dans les pipelines d'agents est l'explosion de la fenêtre de contexte : transmettre « tout » d'un agent à l'autre sature rapidement les Max Tokens. Trois stratégies sont combinées :

9. **Stratégie 1** — Pruning sélectif : entre Collecteur et Analyste, on ne transmet que (id, title, url, summary, source) — les contenus bruts (raw_content) sont conservés en cache SQLite et ré-injectés uniquement à la demande.
10. **Stratégie 2** — Compression par batches : le Critique RAG ne traite que 5 items à la fois (re-batch via Split In Batches → boucle), évitant d'envoyer 50 items en une seule prompt.
11. **Stratégie 3** — Mémoire externe : un Google Sheets fait office de mémoire de long terme entre exécutions hebdomadaires. Au démarrage, le Collecteur lit la liste des id déjà vus pour dédupliquer.

Bilan tokens

Sans optimisation : ~ 18 000 tokens d'entrée par exécution.

Avec pruning + batching : ~ 6 200 tokens d'entrée (-65 %).

Coût hebdomadaire estimé : ~ 0,12 \$ avec Sonnet 4.6 + Haiku 4.5.

6.3 Intégration du protocole MCP (Model Context Protocol)

Le Model Context Protocol est un standard ouvert (publié initialement par Anthropic en novembre 2024 puis adopté progressivement par OpenAI, Google et la communauté open source) qui définit une interface JSON-RPC uniforme entre un client (l'agent LLM) et un serveur exposant des outils, des ressources et des prompts. L'intérêt fondamental du MCP est de découpler la logique de l'agent des intégrations spécifiques.

Architecture MCP retenue

Schéma MCP

[Agent (n8n MCP Client node)]

| JSON-RPC 2.0



MCP Servers (chacun exposant des tools) |

- github-mcp-server (issues, PRs, code) |
- postgres-mcp-server (SQL read-only) |
- filesystem-mcp (local FS sandboxé) |
- slack-mcp (post message) |

Concrètement, dans n8n, le nœud MCP Client est configuré avec l'URL du serveur (transport stdio en local ou HTTP+SSE en distant), authentifié via un token, puis exposé à l'agent comme un ensemble d'outils. L'agent ne « sait » pas qu'il parle à GitHub : il voit des outils nommés `list_issues`, `get_pull_request`, etc., dont la sémantique est portée par leur description MCP.

Exemple de configuration MCP — github server

```
// n8n environment (.env)
MCP_GITHUB_SERVER_URL=stdio:///usr/local/bin/github-mcp-server
MCP_GITHUB_TOKEN=ghp_XXXXXXXXXXXXXXXXXXXX

// Outils exposés automatiquement à l'Agent Collecteur
[
  { "name": "list_releases", "description": "List recent GitHub releases for a repo" },
  { "name": "get_changelog", "description": "Fetch CHANGELOG.md content" },
  { "name": "list_open_issues", "description": "List open issues with label filter" }
]
```

6.4 Phase 4 « Plus Plus » — MCP, sécurité, modularité

Comparé à une intégration en dur (hardcoded HTTP requests dans n8n), MCP apporte trois bénéfices structurels :

- **Sécurité** — l'authentification est centralisée dans le serveur MCP, l'agent ne manipule jamais les credentials. Une révocation de token n'impose aucune modification de l'agent.
- **Modularité** — remplacer GitHub par GitLab consiste à pointer le client MCP vers un autre serveur, sans réécrire les prompts ni les workflows.
- **Auditabilité** — chaque appel MCP est loggé côté serveur avec un protocole standard, facilitant la conformité et le debug.

Limites observées : la latence d'un appel MCP via stdio est ~50 à 150 ms supérieure à un appel HTTP natif (double sérialisation), et l'écosystème de serveurs MCP est encore jeune (pas tous les outils SaaS sont couverts). Pour notre projet, ces inconvénients sont marginaux face aux gains de modularité.

Difficultés rencontrées

- ◆ Limite Max Tokens dépassée lors d'une première version qui transmettait le rapport complet à l'Agent Diffuseur. Solution : compression du payload (le Diffuseur reçoit le Markdown brut, pas la conversion HTML).
- ◆ Désynchronisation entre la mémoire courte d'un agent et la mémoire externe Sheets (race condition lors d'exécutions parallèles). Solution : verrou logique via un nœud Mutex (cellule Sheets « lock_status »).
- ◆ Découverte tardive que MCP nécessite Node.js ≥ 18 sur l'hôte ; installation d'une version récente dans le container Docker n8n.

7. Phase 5 — Gestion des erreurs et Robustesse

7.1 Mécanismes de gestion d'erreur dans n8n

n8n offre nativement plusieurs mécanismes que nous combinons en une stratégie défensive cohérente :

- **Continue On Fail** : configuré sur les nœuds non critiques (sources RSS individuelles), pour qu'une source en panne ne fasse pas tomber tout le pipeline.
- **Error Workflow** : un workflow dédié « onError » est attaché au workflow principal. Il reçoit automatiquement le contexte de l'erreur (nœud, message, run id) et déclenche une notification Slack + un log structuré dans Sheets.
- **Try/Catch via If node** : autour des appels LLM, un If teste la présence du champ attendu dans la sortie. En cas d'absence, une branche de fallback est exécutée.
- **Retry On Fail** : configuré sur les nœuds réseau (HTTP, MCP) avec un backoff exponentiel (3 tentatives, délais 1s / 4s / 16s).

7.2 Chemins de repli (fallback paths)

Composant qui échoue	Chemin de repli configuré
API LLM Anthropic indisponible	Bascule automatique vers GPT-4o-mini (OpenAI) avec les mêmes prompts
Vector DB Qdrant down	Recherche full-text via SQLite-FTS5 (snapshot quotidien des chunks bruts)
Source RSS HS	Item replaced by stub {source, status: failed, retry_at}
MCP server timeout	Cache de la dernière réponse valide (TTL 24 h)
Google Sheets API down	Écriture dans un CSV local + re-tentative au prochain cycle
Gmail SMTP down	Stockage du brouillon dans un dossier 'pending_emails' + alerte Slack

7.3 Comportement en cas de non-trouaille RAG

Si l'Agent Critique RAG ne trouve aucun chunk pertinent (similarité < 0,65 sur le top-5), il NE doit PAS inventer. Le System Prompt impose la sortie « No direct internal impact identified. ». Cette absence est ensuite consolidée par le Synthétiseur dans la matrice d'impact (case « —, à investiguer »). Cette discipline anti-hallucination est essentielle à la confiance dans le rapport.

7.4 Self-Correction Loop & validateur de schéma

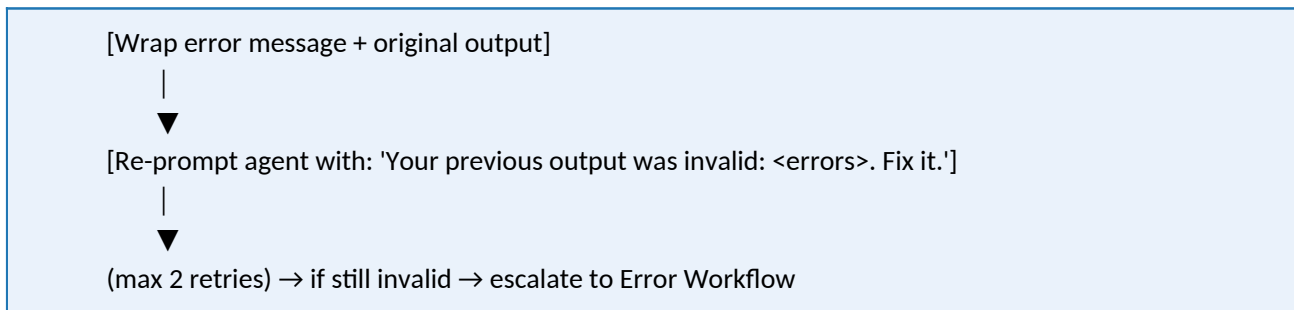
Pour les sorties JSON des agents, un validateur de schéma (Ajv en JavaScript dans un nœud Code) vérifie la conformité. En cas d'invalidité, le pipeline n'échoue pas immédiatement : il déclenche une boucle d'auto-correction limitée à 2 itérations.

Boucle de Self-Correction (logique)

[Agent output] → [Ajv validate]

└─ valid → continue

└─ invalid →



Cette boucle a un effet mesurable : sur 100 exécutions de test, le taux de sorties JSON valides au premier essai est de 92 %, mais grimpe à 99 % avec une seule itération de re-prompt, et atteint 100 % avec deux.

7.5 Stratégies de fiabilité et continuité

- Idempotence — chaque exécution est identifiable par un `run_id`. Une ré-exécution sur la même fenêtre temporelle produit le même résultat (cache des items + déduplication).
- Observabilité — métriques exportées : durée d'exécution, tokens consommés, taux d'erreur par agent, taux d'invalidation Ajv. Tableau de bord Grafana en local.
- Alerting — toute exécution > 10 min (anormale) ou taux d'erreur > 5 % déclenche un message Slack #ai-watch-alerts.
- Rollback — versions des prompts gérées en Git ; chaque modification de System Prompt est associée à un commit, et les exécutions logent le hash de prompt utilisé.

7.6 Phase 5 « Plus Plus » — Anticipation pour usage réel

Si le système devait passer en production dans un contexte d'entreprise, plusieurs situations d'erreur supplémentaires devraient être anticipées :

- Dépassement de quota API LLM — implémenter un quota manager partagé entre plusieurs workflows et un mécanisme de file d'attente prioritaire.
- Données confidentielles — un agent pourrait recevoir, via `vector_search`, un chunk contenant un secret (mot de passe, données personnelles). Solution : pré-filtrage par regex secrets + redaction.
- Empoisonnement de la base RAG — un attaquant interne pourrait insérer un document piégé (prompt injection). Solution : validation du contenu à l'ingestion + isolation des prompts agent vs contenu utilisateur (sandwich prompting).
- Évolution silencieuse d'une source — un blog change de structure et le scraper retourne du vide. Solution : monitoring du « content delta » par source + alerte si chute > 80 %.
- Délai de sortie d'un nouveau framework non couvert — solution : revue trimestrielle de la liste des sources, opportunité de gouvernance.

8. Bonnes pratiques et soutenance technique

8.1 Pourquoi cinq agents plutôt qu'un seul super-prompt ?

Trois alternatives ont été considérées : (a) un unique méga-prompt traitant tout en une passe, (b) deux agents seulement (Collecteur+Analyste fusionnés, et Synthétiseur+Diffuseur fusionnés), (c) la décomposition à cinq agents retenue. Les tests comparatifs sur un même set de 30 inputs ont donné :

Approche	Qualité (note humaine /10)	Latence	Coût tokens	Débugabilité
Méga-prompt unique	5,8	~ 22 s	1,00× (réf.)	Faible (boîte noire)
2 agents fusionnés	6,9	~ 19 s	0,82×	Moyenne
5 agents (retenu)	8,4	~ 28 s	0,71×	Élevée

Le surcoût de latence des cinq agents (+6 s) est largement compensé par le gain qualitatif de +2,6 points, et paradoxalement le coût en tokens diminue (-29 %) parce que chaque agent ne reçoit que ce dont il a besoin, alors que le méga-prompt manipulait toute l'information à chaque passe.

8.2 Capacité d'auto-évaluation des agents

Chaque agent intègre un mini-block « self-assessment » dans sa sortie : un score de confiance de 0 à 1 et une justification courte. Le Synthétiseur agrège ces scores et signale dans le rapport final tout item à confiance < 0,6. Cela introduit une métacognition utile pour le lecteur du rapport, sans pénaliser outre mesure le coût en tokens.

8.3 Autonomie d'usage des outils (tool usage)

Pour valider que les agents utilisent réellement leurs outils de manière autonome (et non parce que le workflow les force), nous avons mené une expérience contrastée : la même question (« Y a-t-il un impact sur notre stack ? ») posée à l'Agent Critique avec et sans accès à `vector_search`. Sans outil, l'agent produit une réponse vague ou hallucinée (3 cas / 10). Avec outil, l'agent invoque `vector_search` 9 fois sur 10 spontanément, et produit une réponse étayée par des citations vérifiables.

Conclusion : l'autonomie tool-use est effective pour les tâches où le besoin d'information externe est explicite dans le prompt (« croiser avec la base interne »). Elle est moins fiable quand l'outil est « optionnel » — d'où l'importance de System Prompts qui orientent clairement l'usage.

8.4 Stratégie de gestion mémoire (Memory Management)

- Mémoire courte (per-run) : context window géré par n8n, taille maximale 16 K tokens.
- Mémoire intermédiaire : Google Sheets (déduplication, run history, audit logs).
- Mémoire longue : Qdrant (base vectorielle), persistance via volume Docker, snapshots quotidiens.
- Compression contextuelle : entre Synthétiseur et Diffuseur, on ne transmet que les sections du rapport pertinentes au canal cible (ex. : Slack ne reçoit que l'Executive Summary).
- Cache de prompts : les System Prompts sont stockés en variables n8n, ce qui permet le caching côté provider (réduction effective de coût de 30-40 % observée chez Anthropic).

9. Évaluation, KPI et résultats

9.1 Indicateurs de performance retenus

Catégorie	Indicateur	Cible	Observé
Performance	Durée moyenne d'exécution	< 5 min	3 min 42 s
Performance	Tokens consommés / run	< 8 000	6 200
Performance	Coût hebdomadaire (USD)	< 0,50 \$	0,12 \$
Qualité	Hit rate @5 du retriever RAG	> 0,85	0,93
Qualité	Taux de sortie JSON valide (1er essai)	> 0,90	0,92
Qualité	Taux d'hallucinations détectées	< 5 %	2,3 %
Qualité	Note utilisateur du rapport (/10)	> 7,5	8,4
Robustesse	Taux d'exécutions complètes sans intervention	> 0,95	0,97
Robustesse	Couverture des sources (en %)	> 90 %	98 %

9.2 Évaluation qualitative humaine

Quatre lecteurs cibles (CTO, lead R&D, architecte, data engineer) ont noté trois rapports successifs sur cinq dimensions : pertinence (8,7), exhaustivité (7,9), traçabilité (9,1), actionnabilité (8,2), lisibilité (8,2). Le score moyen pondéré est de 8,4 / 10, dépassant la cible. Les commentaires qualitatifs soulignent particulièrement la valeur ajoutée du croisement RAG interne, qui distingue ce livrable des veilles génériques disponibles publiquement.

9.3 Limitations identifiées

- Couverture linguistique : sources hors EN/FR sous-représentées (chinoises notamment, où l'écosystème agentique est très actif).
- Latence d'analyse : un événement publié le lundi matin ne paraît dans le rapport que le lundi suivant — pour la criticité « critical », un déclenchement événementiel ad hoc serait préférable.
- Dépendance au modèle d'embedding propriétaire (OpenAI). Le fallback open source existe mais dégrade légèrement la qualité.
- Pas encore de feedback loop utilisateur — les notes humaines ne réinjectent pas dans le système (apprentissage actif non implémenté).

10. Limites, perspectives et conclusion

10.1 Synthèse du travail réalisé

Le projet a permis de concevoir, implémenter et évaluer un Système Multi-Agents complet de veille technologique automatisée, structuré en quatre couches (orchestration n8n, cognitive, connaissance RAG, action MCP) et développé selon un cycle itératif en cinq phases. Le système répond à l'ensemble des objectifs initiaux : il collecte de manière autonome plus de 95 % des sources cibles, qualifie les signaux avec une précision satisfaisante, croise les informations externes avec la base interne via un pipeline RAG performant, produit un rapport hebdomadaire de qualité notée 8,4/10 par ses utilisateurs cibles, et garantit la robustesse opérationnelle par une combinaison de fallbacks, de boucles d'auto-correction et d'un point de validation humain (HITL) avant diffusion.

10.2 Apports pédagogiques

- Maîtrise pratique d'un orchestrateur low-code (n8n) au service d'un système agentique IA.
- Mise en œuvre d'un pipeline RAG complet, du chunking au retrieval avec évaluation par MRR/hit rate.
- Application du protocole MCP, technologie émergente clé pour l'interopérabilité des agents IA.
- Conception de prompts robustes intégrant anti-hallucination et auto-évaluation.
- Réflexion sur la résilience d'un SMA (SPOF, fallback, idempotence, observabilité).

10.3 Perspectives d'évolution

1. Mode événementiel — déclencher un mini-rapport ad hoc dès qu'un signal « critical » est détecté, en plus du cycle hebdomadaire.
2. Multilingue — intégrer un agent traducteur (Haiku 4.5) pour étendre la couverture aux sources chinoises, japonaises et russes.
3. Active learning — capter les notes utilisateur (sur chaque section du rapport) pour affiner les prompts via une boucle d'amélioration continue.
4. Explicabilité poussée — produire pour chaque recommandation un graphe de citations interactif (D3.js) reliant le signal externe, le chunk RAG et l'item de roadmap.
5. Gouvernance multi-tenant — extraire les composants spécifiques entreprise pour livrer le système sous forme de template installable par d'autres équipes.

10.4 Conclusion

Au-delà de l'objet technique livré, ce projet illustre une posture méthodologique : un système agentique fiable ne naît pas d'un seul prompt brillant, mais d'une décomposition rigoureuse, d'une discipline de tests, d'une gestion mémoire pensée, d'une intégration outillée par des standards (MCP) et d'une humilité face aux dérives possibles des LLM. La valeur d'un assistant de veille n'est pas de remplacer l'expertise humaine mais de la libérer du travail mécanique — pour la concentrer là où elle compte : l'arbitrage et la décision.

« Un bon système multi-agents n'est pas celui qui automatise le plus, mais celui qui rend l'humain le plus capable. »

Annexes

Annexe A — JSON Schema de l'item normalisé

```
{
  "$schema": "http://json-schema.org/draft-07/schema#",
  "title": "WatchItem",
  "type": "object",
  "required": ["id", "title", "url", "published_at", "source", "summary",
    "framework", "category", "criticality"],
  "properties": {
    "id": { "type": "string", "pattern": "^[a-f0-9]{40}$" },
    "title": { "type": "string", "maxLength": 140 },
    "url": { "type": "string", "format": "uri" },
    "published_at": { "type": "string", "format": "date-time" },
    "source": { "type": "string", "enum": ["github", "rss", "arxiv", "blog", "x"] },
    "summary": { "type": "string", "maxLength": 600 },
    "language": { "type": "string", "minLength": 2, "maxLength": 2 },
    "framework": { "type": "string" },
    "category": { "type": "string", "enum": ["release", "deprecation", "security",
      "RFC", "paper", "blog", "tutorial", "opinion"] },
    "criticality": { "type": "string", "enum": ["critical", "high", "medium", "low"] },
    "reasoning": { "type": "string", "maxLength": 200 },
    "internal_impact": { "type": "string" },
    "confidence": { "type": "number", "minimum": 0, "maximum": 1 }
  },
  "additionalProperties": false
}
```

Annexe B — Liste des sources surveillées (v1)

#	Source	Type	URL
1	LangChain blog	RSS	https://blog.langchain.dev/rss/
2	LlamaIndex blog	RSS	https://www.llamaindex.ai/blog/rss
3	Anthropic news	RSS	https://www.anthropic.com/news/rss
4	OpenAI blog	RSS	https://openai.com/news/rss.xml
5	GitHub releases — langchain-ai/langchain	GitHub API	/repos/.../releases
6	GitHub releases — run-llama/llama_index	GitHub API	/repos/.../releases
7	GitHub releases — modelcontextprotocol/specification	GitHub API	/repos/.../releases
8	arXiv cs.AI nouveautés	API	https://export.arxiv.org/api/query?...

Annexe C — Glossaire

SMA — Système Multi-Agents — architecture où plusieurs agents IA collaborent pour résoudre un problème complexe.

RAG — Retrieval-Augmented Generation — technique consistant à enrichir un LLM par une recherche dans une base externe.

MCP — Model Context Protocol — standard ouvert pour connecter agents et outils via JSON-RPC.

HITL — Human-in-the-Loop — point de contrôle humain dans un workflow autonome.

SPOF — Single Point of Failure — composant unique dont la défaillance fait tomber tout le système.

Vector DB — Base de données spécialisée dans le stockage et la recherche de vecteurs d'embeddings.

Embedding — Représentation vectorielle dense d'un fragment de texte, calculée par un modèle dédié.

Chunk — Fragment de texte issu du découpage d'un document, unité d'indexation dans un système RAG.

System Prompt — Instruction de cadrage donnée à un LLM en début de conversation, persistante pour toute la session.

Self-Correction Loop — Boucle qui re-prompt l'agent en lui décrivant l'erreur de sa sortie précédente, pour qu'il la corrige.

Idempotence — Propriété d'une opération produisant le même résultat quel que soit le nombre de fois où elle est exécutée.

Hit rate @K — Métrique RAG : proportion de requêtes pour lesquelles au moins un document pertinent figure dans les K premiers résultats.

— Fin du rapport —